

**LAPORAN PRAKTIKUM
APLIKASI BERBASIS PLATFORM
PERTEMUAN 9&10**



**DISUSUN OLEH :
AHMAD TITANA NANDA PRAMUDYA
2311102042**

**DOSEN PENGAMPU :
Dimas Fanny Hebrasianto Permadi, S.ST., M.Kom**

Asisten Praktikum :
Muhammad Afiq Tamamul Wafa
Rangga Pradarrell Fathi
**LABORATORIUM HIGH PERFORMANCE
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026**

MODUL 9&10

A. Dasar Teori

1. Provider (State Management)

Provider adalah pustaka state management yang direkomendasikan oleh Flutter untuk memisahkan logika antarmuka pengguna (UI) dari logika bisnis. Dengan menggunakan Provider, pembaruan status data seperti menambah atau menghapus daftar tugas yang dapat dilakukan secara efisien tanpa harus merender ulang seluruh halaman, melainkan hanya pada komponen widget yang membutuhkan perubahan data saja.

2. Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging (FCM) adalah layanan perpesanan lintas platform gratis dari Google yang memungkinkan pengiriman notifikasi secara real-time dan andal. Pada aplikasi ini, FCM digunakan untuk menangkap payload pesan jarak jauh dari Firebase Console dan menampilkannya kepada pengguna.

B. TUGAS

1. Source Code Aplikasi To-Do List (Provider & FCM)

- File lib/main.dart

```
1 import 'package:flutter/material.dart';
2 import 'package:firebase_core/firebase_core.dart';
3 import 'package:firebase_messaging/firebase_messaging.dart';
4 import 'package:provider/provider.dart';
5 import 'firebase_options.dart';
6 import 'providers/todo_provider.dart';
7 import 'screens/home_screen.dart';
8
9 //pesan saat aplikasi berjalan di background
10 Future<void> _firebaseMessagingBackgroundHandler(RemoteMessage message) async {
11   print("Handling a background message: ${message.messageId}");
12 }
13
14 void main() async {
15   WidgetsFlutterBinding.ensureInitialized();
16   await Firebase.initializeApp(
17     options: DefaultFirebaseOptions.currentPlatform,
18   );
19   FirebaseMessaging.onBackgroundMessage(_firebaseMessagingBackgroundHandler);
20
21   runApp(
22     MultiProvider(
23       providers: [
24         ChangeNotifierProvider(create: (_) => TodoProvider()),
25       ],
26       child: const MyApp(),
27     ),
28   );
29 }
30
31 class MyApp extends StatelessWidget {
32   const MyApp({super.key});
33
34   @override
35   Widget build(BuildContext context) {
36     return MaterialApp(
37       title: 'To-Do FCM App',
38       theme: ThemeData(
39         primarySwatch: Colors.blue,
40         useMaterial3: true,
41       ),
42       home: const HomeScreen(),
43       debugShowCheckedModeBanner: false,
44     );
45   }
46 }
```

- File lib/screens/home_screen.dart

```

1 import 'package:flutter/material.dart';
2 import 'package:provider/provider.dart';
3 import 'package:firebase_messaging/firebase_messaging.dart';
4 import '../providers/todo_provider.dart';
5
6 class HomeScreen extends StatefulWidget {
7   const HomeScreen({super.key});
8
9   @override
10  State<HomeScreen> createState() => _HomeScreenState();
11 }
12
13 class _HomeScreenState extends State<HomeScreen> {
14   final TextEditingController _taskController = TextEditingController();
15
16   @override
17   void initState() {
18     super.initState();
19     _setupFCM();
20   }
21
22   void _setupFCM() async {
23     FirebaseMessaging messaging = FirebaseMessaging.instance;
24
25     // Meminta izin notifikasi (penting untuk iOS / Android 13+)
26     await messaging.requestPermission();
27
28     // Dapatkan token untuk testing pengiriman spesifik di Firebase Console
29     String? token = await messaging.getToken();
30     print("FCM Token Anda: $token"); // Cek terminal untuk copy token ini
31
32     // Listener untuk notifikasi saat aplikasi dibuka (foreground)
33     FirebaseMessaging.onMessage.listen((RemoteMessage message) {
34       if (message.notification != null) {
35         ScaffoldMessenger.of(context).showSnackBar(
36           SnackBar(
37             content: Text('Notifikasi Masuk: ${message.notification?.title} - ${message.notification?.body}'),
38             backgroundColor: Colors.green,
39             duration: const Duration(seconds: 4),
40           ),
41         );
42       }
43     });
44   }
45
46   @override
47   Widget build(BuildContext context) {
48     final todoProvider = Provider.of<TodoProvider>(context);
49
50     return Scaffold(
51       appBar: AppBar(
52         title: const Text('My Life'),
53         actions: [
54           IconButton(
55             icon: const Icon(Icons.delete_sweep, color: Colors.red),
56             onPressed: () => todoProvider.clearAllTasks(),
57             tooltip: 'Hapus Semua Tugas',
58           ),
59         ],
60     ),

```

```

61   body: todoProvider.tasks.isEmpty
62     ? const Center(child: Text('Belum ada tugas. Tambahkan sekarang!'))
63     : ListView.builder(
64       itemCount: todoProvider.tasks.length,
65       itemBuilder: (context, index) {
66         return Card(
67           margin: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
68           child: ListTile(
69             leading: const Icon(Icons.task_alt, color: Colors.blue),
70             title: Text(todoProvider.tasks[index]),
71           ),
72         );
73       },
74     ),
75   floatingActionButton: FloatingActionButton(
76     onPressed: () => _showAddTaskDialog(context, todoProvider),
77     child: const Icon(Icons.add),
78   ),
79 );
80 }
81
82 void _showAddTaskDialog(BuildContext context, TodoProvider provider) {
83   showDialog(
84     context: context,
85     builder: (context) {
86       return AlertDialog(
87         title: const Text('Tambah Tugas Baru'),
88         content: TextField(
89           controller: _taskController,
90           decoration: const InputDecoration(hintText: 'Contoh: Mengerjakan Modul 12'),
91           autofocus: true,
92         ),
93         actions: [
94           TextButton(
95             onPressed: () {
96               _taskController.clear();
97               Navigator.pop(context);
98             },
99             child: const Text('Batal'),
100          ),
101          ElevatedButton(
102            onPressed: () {
103              provider.addTask(_taskController.text);
104              _taskController.clear();
105              Navigator.pop(context);
106            },
107            child: const Text('Simpan'),
108          ),
109        ],
110      );
111    },
112  );
113 }

```

- File lib/providers/todo_provider.dart

```
1 import 'package:flutter/material.dart';
2
3 class TodoProvider extends ChangeNotifier {
4   final List<String> _tasks = [];
5
6   List<String> get tasks => _tasks;
7
8   void addTask(String task) {
9     if (task.trim().isEmpty) {
10       _tasks.add(task);
11       notifyListeners(); // Memperbarui UI
12     }
13   }
14
15   void clearAllTasks() {
16     _tasks.clear();
17     notifyListeners(); // Memperbarui UI
18   }
19 }
```

2. Hasil Output Aplikasi dan Penjelasan

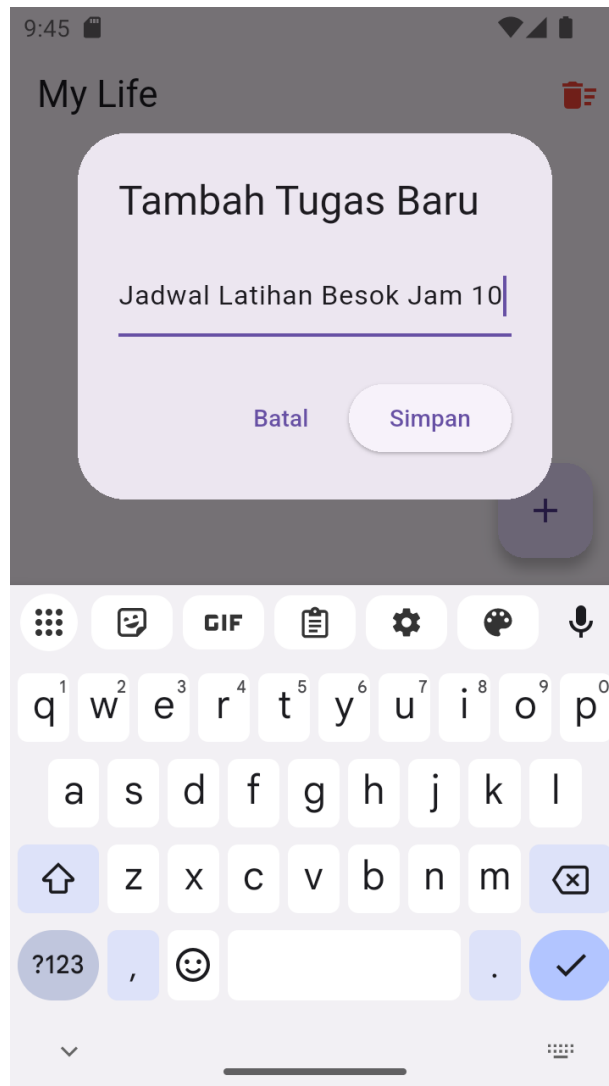
- Output 1: Tampilan Utama Aplikasi



Penjelasan:

Gambar ini mendemonstrasikan fitur empty state pada aplikasi To-Do List "My Life". Berkat State Management Provider, antarmuka secara otomatis mendeteksi ketiadaan data dan meresponsnya dengan menampilkan pesan "Belum ada tugas..." beserta tombol "+" di sudut bawah untuk mulai menambahkan tugas baru.

- Output 2: Tampilan Penambahan Tugas

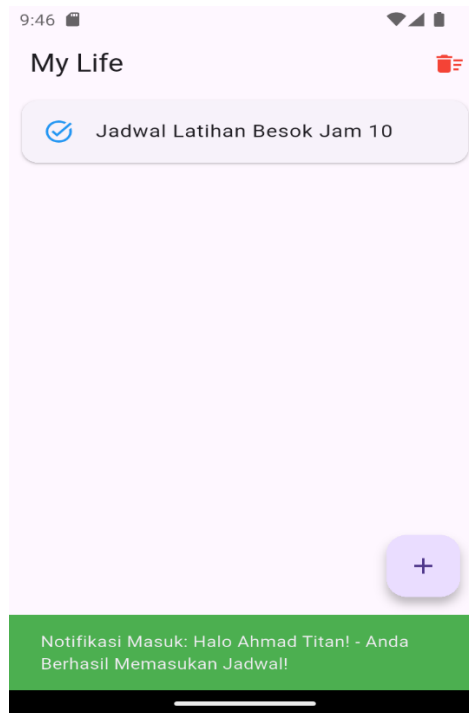


Penjelasan:

Gambar ini memperlihatkan antarmuka AlertDialog interaktif yang muncul ketika pengguna menekan tombol Floating Action Button (+). Melalui kotak dialog ini, pengguna dapat mengetikkan deskripsi tugas baru, seperti yang terlihat pada gambar. Ketika tombol "Simpan" ditekan, fungsi penambahan data di dalam TodoProvider akan langsung dieksekusi sehingga sistem dapat memperbarui tampilan antarmuka secara reaktif. Hal ini memungkinkan tugas

yang baru saja dibuat untuk langsung muncul di daftar utama tanpa mengharuskan pengguna memuat ulang (restart) halaman aplikasi.

- Output 3: Tampilan Penerimaan Notifikasi (FCM)



Penjelasan:

Gambar di atas merupakan bukti keberhasilan pengujian integrasi Firebase Cloud Messaging (FCM) pada aplikasi. Ketika pesan uji coba dikirimkan dari Firebase Console menggunakan Registration Token spesifik milik perangkat emulator, aplikasi yang sedang dalam keadaan terbuka (foreground) berhasil menangkap payload pesan tersebut dengan baik. Selanjutnya, sistem langsung mengeksekusi pesan yang diterima dan menampilkannya seketika kepada pengguna dalam bentuk Snackbar berwarna hijau di bagian bawah layar.